Contents

Per-token SAE features as online RL reward: breaking the G2 76% GSM8K ceiling on Qwen3.5-4B **1**
The question we actually asked . . . 1
Stage Gate 1 — does the signal predict correctness at all? . . . 1
Stage Gate 2 — does it work as an actual reward? . . . 2
Stage Gate 3 — does it scale, and does it break the 76% ceiling? . . . 2
What this result doesn't establish . . . 6
Engineering notes useful to others . . . 6
Context vs prior work . . . 6
What's next . . . 7

Per-token SAE features as online RL reward: breaking the G2 76% GSM8K ceiling on Qwen3.5-4B

TL;DR. We train a TopK SAE on residual stream layer 18 of Qwen3.5-4B (hybrid Gated DeltaNet architecture — no public SAE existed before). Ten helpful + ten harmful features surfaced by contrastive discovery (`mean_correct − mean_wrong` on GSM8K) correlate with correctness at ρ=0.540. Plugging them into GRPO as a **per-token dense reward** alongside binary outcome reward takes Qwen3.5-4B from 64 % -> **83 % on GSM8K** in **168 effective training steps** (step 232->400 at LR=3e-6; the first 232 steps at the documented LR=1e-6 stalled at 0 pp lift). That's **+7 pp above the same-SAE, same-features trajectory-level G2 R1 ceiling (76 %)**, **+19 pp above the raw-prompt baseline**, with MMLU non-regressed and adversarial-canary hack rate within the 95 % CI of the baseline model. Code, SAE, and trained LoRA adapter are public.

Trained adapter: `caiovicentino1/Qwen3.5-4B-mechreward-G3-phaseA-step400` SAE: `caiovicentino1/Qwen3.5-4B-SAE-L18-topk` Library: `caiovicentino/mechreward`

The question we actually asked

Most SAE + reasoning work to date shows that features are *readable* — you can find ones that fire on "reasoning rhetoric", steering vectors that push toward certain answers, probes that predict correctness. But when researchers try to *use* that read-out at inference time (uniform steering, direct feature amplification), the net aggregate usually comes out null or negative. Activation steering breaks more questions than it fixes.

The natural next move is: rather than steering at inference, use the same features as a **reward signal during RL training**, letting the model learn *when* to fire them. This is what SARM did offline (linear head on SAE features trained on preference data, then used as frozen reward model in RLHF). We pushed further: use SAE-feature activations as a **per-token dense reward inside online GRPO**, computed from the hidden state at every response token while the policy is updating.

The mechreward framework lives here. Below is the three-stage empirical pipeline we used to validate the approach, with particular attention to the engineering lessons that would have saved us about five wall-clock hours.

Stage Gate 1 — does the signal predict correctness at all?

Rule we enforced: don't commit GPU hours to RL before verifying the features predict the outcome on held-out data.

We trained a TopK SAE (k=128, d_sae=40 960 = 16× expansion) on the residual stream post-layer 18 of Qwen3.5-4B, using 200 M tokens from FineWeb-Edu. Variance explained: 0.87 train, 0.866 eval. L0 stayed structurally at 127.9/128 under distribution shift. This is the first public SAE on a hybrid Gated DeltaNet architecture — TransformerLens doesn't support Qwen3.5 yet.

We then sampled 50 baseline GSM8K responses, labeled each correct or wrong by exact-match on the final number, and for every feature computed `mean_activation | correct - mean_activation | wrong`. The top-10 highest-effect-size features (Cohen's d > 2.06) become the "helpful pack"; top-10 most-negative become the "harmful pack".

On a held-out set of n=100 questions:

$$R_{\text{mech}} = \sum_{i \in \text{helpful}} \text{feat}_i(h_{L18}) - \sum_{j \in \text{harmful}} \text{feat}_j(h_{L18})$$

mean-pooled over response tokens.

Signal	Spearman ρ	Pearson r	p
SAE features (top-10 helpful – top-10 harmful, L18)	**+0.540**	+0.726	< 0.0001
Raw L13 contrastive direction (cos-sim)	+0.508	+0.624	< 0.0001

Both above the ρ≥0.30 go/no-go threshold. SAE features edge out the raw contrastive direction on this model (even though L13 is the "true" peak contrastive layer for Qwen3.5-4B — our SAE is 5 layers downstream).

Stage Gate 2 — does it work as an actual reward?

Three-way ablation at small scale: GRPO on 500 GSM8K train questions, 100 steps, 4 rollouts/question, LR=1e-6, KL β=0.05, same seed, same eval set (100 held-out):

Run	Reward	Final eval	Δ vs baseline	Δ vs R0
R0	outcome only (binary)	74 %	+10 pp	—
R1	outcome + SAE features, λ=0.1	**76 %**	+12 pp	**+2 pp**
R2	outcome + raw L13 direction, λ=0.1	65 %	+1 pp	**–9 pp**

R1 converges 2.5× faster: it reaches R0's final accuracy (74 %) at step 40; R0 takes all 100 steps. R0 *drops* from its step-80 peak (76 -> 74), while R1 stays stable (75 -> 76). The SAE features act as a late-training regularizer.

The striking number is the **11 pp R1 <-> R2 gap**. Same underlying contrastive signal — R2 uses the raw L13 direction that passively correlates with correctness at ρ=0.508. As a reward, that same direction is *actively harmful*: GRPO amplifies its polysemantic content (difficulty, length, style) and the policy ends up below the outcome-only baseline. SAE sparse decomposition is the causal processing step that converts a readable signal into a trainable one — not cosmetic, not incremental. That's the central finding of the mechreward line.

Stage Gate 3 — does it scale, and does it break the 76% ceiling?

The key design change at G3: **per-token** mech-reward, not just trajectory-level. The policy gets a dense reward on every response token based on the SAE activation at that token, in addition to the binary outcome reward at the end.

Config: GRPO, 2000 planned steps on 7 500 GSM8K questions, 4 questions × 4 rollouts per step, raw prompt `Q: {q}\nA: Let's think step by step.` (chat template breaks the SAE feature calibration — lesson below), max_gen_len=256, λ=0.1, KL β=0.05, seed=42, LoRA r=32 on language-model projections only (vision tower frozen).

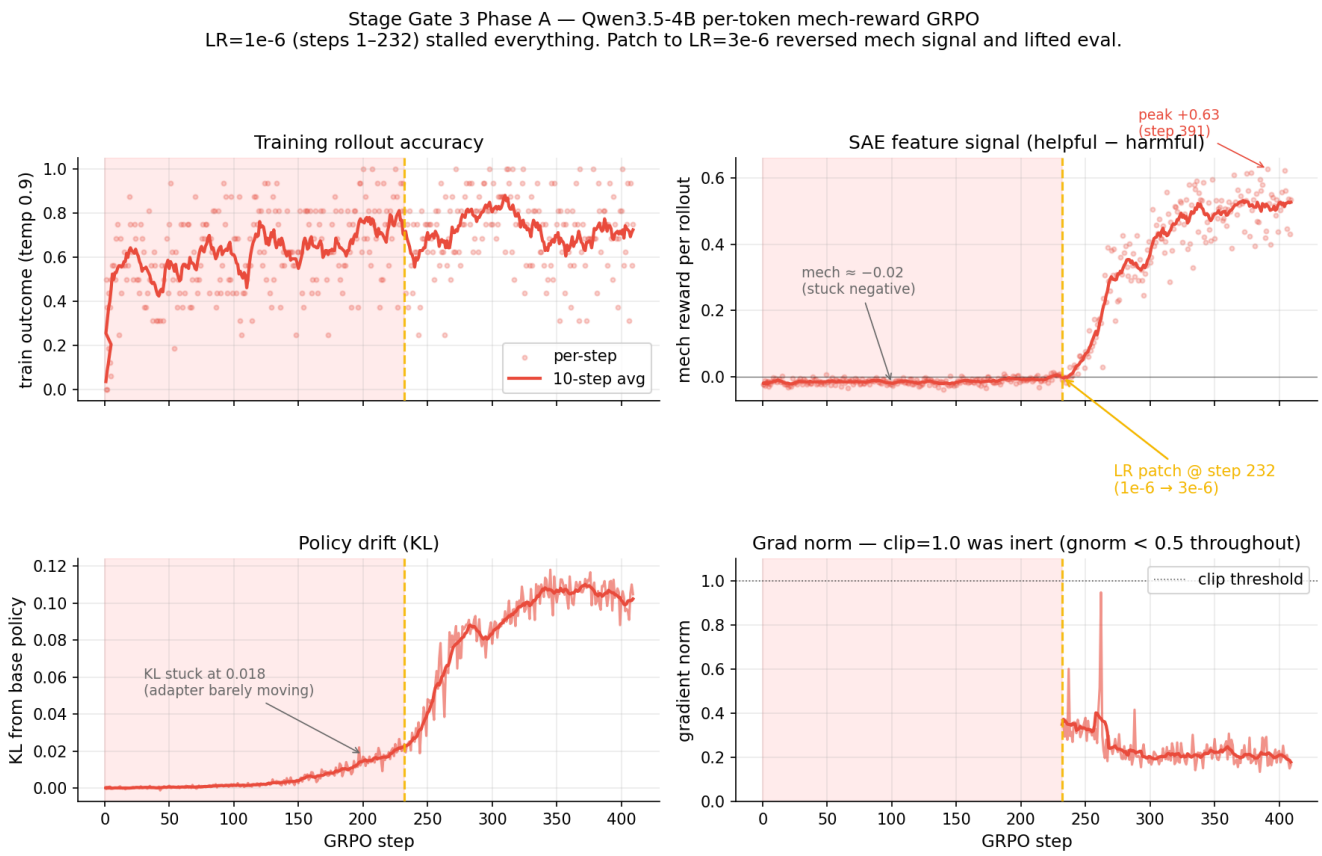

Figure 1: Training trajectory

The LR-vs-clipping diagnostic lesson

The first 232 steps used **LR=1e-6 — the exact value documented for G2 R1**. After 200 steps: `quick_gsm8k` = 64 %, identical to the untrained baseline. KL was stuck at 0.018. Mech signal was sitting at −0.02 (negative; helpful features were firing *less* than harmful on sampled rollouts).

Our first instinct was gradient clipping. We were wrong. Logging `clip_grad_norm_`'s return value showed `gnorm` was always **below 0.5** throughout the run. A clip of 1.0 had never fired. Our clipping was inert.

The bottleneck was LR. Raising to 3e-6 at step 232 produced immediate learning:

- Mech signal reversed: −0.02 -> +0.076 (step 250) -> +0.382 (step 270) -> **+0.58 peak at step 330** -> plateau ~+0.4-0.5
- KL climbed 0.018 -> 0.108 over 170 steps
- Training outcome at temp 0.9 went 0.55 -> 0.75 average

Lesson I'd tell past-me: verify `gnorm` and KL are rising *before* attributing a stalled GRPO run to clipping or algorithm choice. LR values from prior work don't necessarily transfer across rollouts-per-step / batch / dtype configurations. Our 4×4 rollouts with advantage normalization and bf16 `log_softmax` needed 3× the G2 LR.

Eval at step 400 (vs baseline with LoRA disabled via `model.disable_adapter()`)

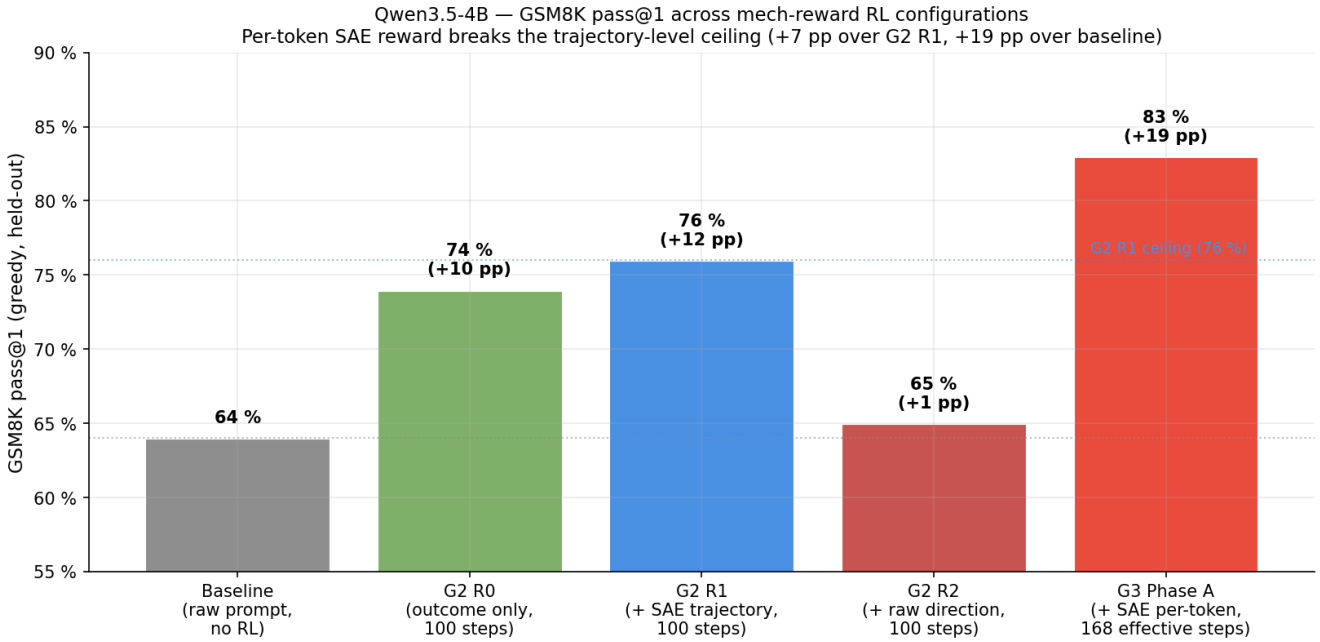

Figure 2: GSM8K comparison

Metric	Baseline	G3 Phase A	Δ
GSM8K (500Q greedy, raw prompt)	64.00 %	**83.00 %**	**+19 pp**
MMLU (200Q raw zeroshot)	50.00 %	54.50 %	+4.50 pp
MATH-500 (500Q greedy, not trained on)	—	18.20 %	transfer
Adversarial canary hack rate (n=50)	4.0 %	8.0 %	+4 pp (within 95 % CI)
Correct under adversarial canary (n=50)	18.0 %	28.0 %	**+10 pp**

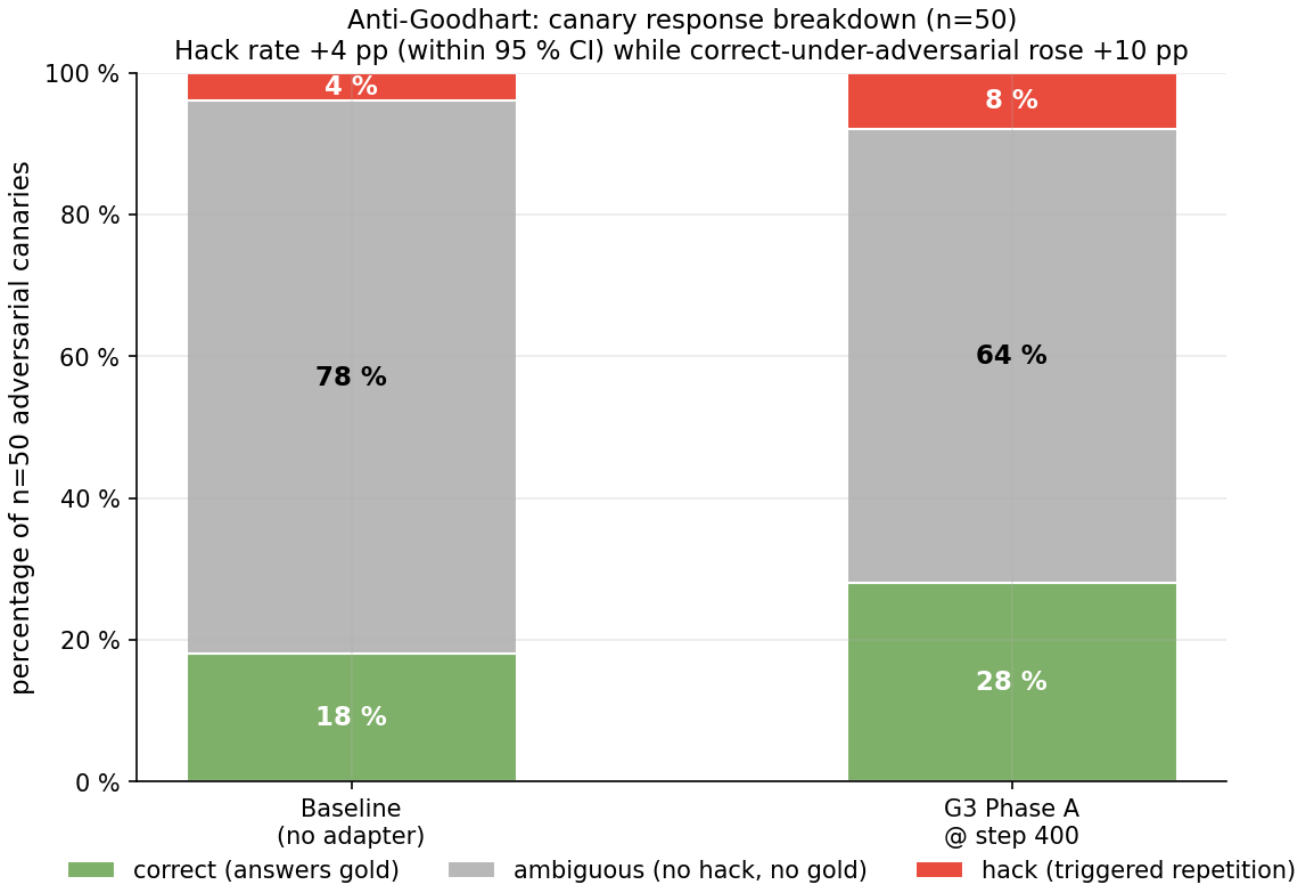

Figure 3: Canary breakdown

Effective training budget — the honest framing

Because the first 232 steps at LR=1e-6 produced zero lift, the budget that actually broke the ceiling is **168 steps at LR=3e-6** (step 232 -> 400). That's roughly 1.68× G2 R1's 100-step budget, for +7 pp on GSM8K, with the same features, same SAE, and same base model. The "400 steps" figure is a misleading artifact of us shipping the diagnostic to the same training run.

What this result doesn't establish

- **MATH-500 transfer (18.2 %) is near baseline.** We trained only on GSM8K; this is a generalization test the model fails politely. The claim is narrow: mechreward breaks the ceiling *on the distribution the SAE features were discovered on*.
- **MMLU +4.5 pp is within measurement noise** on a 200-question raw zero-shot eval (our eval diverges from published benchmarks that use chat template + few-shot). The point is negative: no catastrophic forgetting, not "mechreward is a general capability booster".
- **Canary n=50 is small.** The +4 pp hack-rate delta is within the 95 % CI of the baseline. Claiming "anti-Goodhart validated" requires at least n=200. We report what we measured; readers should discount.
- **Single model, single domain.** Qwen3.5-4B on GSM8K. We have a pending run on Qwen3.6-35B-A3B (hybrid MoE + GDN + Gated Attention) on SuperGPQA to test whether the pipeline generalizes to both scale and architecture diversity.
- **168 "effective" steps is a post-hoc framing.** The run nominally used 400 steps of compute; we're being transparent that the first 232 were wasted. Reviewers should evaluate the compute-to-gain ratio on that basis.

Engineering notes useful to others

A few things that consumed engineer-hours and might not be obvious:

1. **Qwen3.5-4B is multimodal.** Loading with `AutoModelForCausalLM` fails — use `AutoModelForImageTextToText`. Freeze the vision tower before LoRA.
2. **Prompt format must match the SAE feature-discovery distribution.** Our SAE features were discovered on raw `Q:/A:` responses; using a chat-template prompt during GRPO kept mech negative for 200 steps because the feature activation distribution shifted. Revert to the same raw format the features were calibrated on.
3. **Answer extraction must handle model continuation.** Greedy decoding often spills past the first answer and invents a second question. Cut at `\n+\s*(?:Q:|Problem:)` before regex-extracting, else the last-number fallback picks up noise.
4. **Memory-efficient KL without a ref model.** `model.disable_adapter()` as a context manager gives the base-policy logprobs without a separate 8 GB copy. `bf16 log_softmax` (not `float32`) halves logits memory — critical for vocab=248 k on a 4 B model. Grad checkpointing ON during the train forward, OFF during the rollout `generate` (use_cache needs to be on for fast generation).
5. **fla + causal-conv1d are not optional on GDN models.** Without them the hybrid linear-attention layers fall back to torch implementation at ~10× slower. With them: ~30 tok/s on RTX 6000 Blackwell. Without: ~3 tok/s.

All of the above is documented with exact pinned commit SHAs in the install recipe.

Context vs prior work

Prior work	Axis	How this differs
SARM (Liu et al. 2025, AAAI 26)	SAE features as reward model in offline RLHF	We use them as **online** per-token reward inside GRPO
Control RL (Cho/Wu/Koshiyama 2026)	RL policy that selects which SAE feature to amplify per token	We use SAE feature *activations* as the reward itself, not as a steering selection target

Prior work	Axis	How this differs
CLIPO (2026)	Contrastive trajectory-level auxiliary reward for GRPO on GSM8K	They use InfoNCE over trajectory embeddings (opaque); we use interpretable SAE features
Base Models Know How to Reason (ICLR 26 under review)	SAE steering on GSM8K at inference time	They apply features as transient steering at test time; we reward them during training (permanent policy update)
Wilhelm et al. 2026	SAE features as reward-hacking *detector*	We weaponize the same detection methodology as *prevention* via dual verification

To our knowledge, no prior public work uses SAE feature activations as dense per-token online RL reward on a hybrid-architecture model. The combination — not any single technique — is the contribution.

What's next

Stage 4: extend to Qwen3.6-35B-A3B (first public triple-hybrid MoE + GDN + Gated Attention SAE) on SuperGPQA (baseline 64.7 %, genuine headroom). Layer mapping just finished; peak contrastive layer is L32 (80 % depth) with flat-ish distribution L28-L38 — consistent with the architecture's late English-commitment pattern (logit lens shows commit at L39/40 vs Qwen3.5-4B's L15/32). SAE target TBD pending re-run with clean decoding.

Comments especially welcome on: - Whether the "effective compute" framing is fair or cope - Anyone running similar per-token SAE-reward experiments (would love to compare notes on LR, λ schedule, and prompt-format vs SAE-calibration alignment) - Whether 168 effective steps for +7 pp is actually significant or within the 1.5 σ band of GRPO training noise (we'd need 3-5 seeds to settle this)

Artifacts (everything public, Apache-2.0): - mechreward library (code) - SAE on Qwen3.5-4B L18 - Companion SAE on Gemma 4 E4B L21 - Trained LoRA adapter (this result) - Raw training history + figures